

PROGRAMMING II

GENERIC TYPES

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.Y. 2025/2026

GENERIC TYPES



SAME CONTAINER DIFFERENT CONTENT



PARAMETRIC POLYMORPHISM

Methods and parameters are allowed to have a generic type, using **type variables**

- ▶ Write code that is independent from the type of data it handles

Trivial example, the identity function $\lambda x . x$

- ▶ If x has type **int**, $\lambda x . x$ returns the same **int** given as input
- ▶ If x has type **String**, $\lambda x . x$ returns the same **String** given as input

PARAMETRIC POLYMORPHISM (CONT'D)

How can we implement an identity method in Java?

- ▶ Naively, use `Object` as a placeholder
- ▶ `Object identity(Object obj) { return obj; }`

Cumbersome and ugly, we can use **generic types** *// aka generics in Java*

- ▶ Here `<T>` is the type parameter declaration for the method
- ▶ When the method is called you have to provide an actual type for `T`
- ▶ Everything is valid for `T`, except primitive types

```
<T> T identity(T x) { return x; }
```

PARAMETRIC CLASS WITHOUT GENERICS

Class representing pairs of values, without generics

- Use `Object` for the pair values

```
public class Pair {                                1
    private Object first, second;                  2
                                                    3
    public Pair(Object first, Object second) {      4
        this.first = first;                        5
        this.second = second;                      6
    }                                                7
                                                    8
    public Object getFirst() { return first; }       9
    public Object getSecond() { return second; }    10
}                                                    11
```

PARAMETRIC CLASS WITHOUT GENERICS (CONT'D)

You can use the class with different types

```
Pair p = new Pair("one", "two");  
Pair q = new Pair(1, 2); // auto-boxing to Integer
```

1

2

But you have to be very careful with casts

```
String first = (String) p.first();  
String second = (String) q.second(); // runtime ClassCastException
```

1

2

Class representing pairs of values, with generics

```
1 public class Pair<T> { // class type parameter declaration
2     private T first, second;
3
4     public Pair(T first, T second) {
5         this.first = first;
6         this.second = second;
7     }
8
9     public T getFirst() { return first; } // type parameter declaration
10    public T getSecond() { return second; } // not needed here
11 }
```

PARAMETRIC CLASS WITH GENERICS (CONT'D)

You can use the class with different types

```
Pair<String> p = new Pair<String>("one", "two");  
Pair<Integer> q = new Pair<Integer>(1, 2);
```

1

2

But you do not need casts

```
String first = p.first();  
String second = q.second(); // compiler error -- type mismatch
```

1

2

GENERIC TYPE DECLARATION

Generic type declarations may contain more than one type parameter

- ▶ We can make generic references, methods, class, and interfaces

<T1, T2, ... Tk>

Naming conventions

- ▶ Uppercase letters
- ▶ Usually T (ype), E (lement), K (ey), V (alue)

COMPARABLE INTERFACE WITH GENERICS

The `java.lang.Comparable<T>` contains the method

```
int compareTo(T obj);
```

```
public class Student extends Person implements Comparable<Student> { 1
    ... 2
    @Override 3
    public int compareTo(Student other) { // simpler and safer code 4
        return this.matricola - other.matricola; 5
    } 6
} 7
```

ITERABLE AND ITERATOR INTERFACES WITH GENERICS

The `java.lang.Iterable<E>` interface contains the method

```
Iterator<E> iterator();
```

The `java.util.Iterator<E>` interface contains the methods

```
boolean hasNext(); and E next();
```

ITERABLE AND ITERATOR INTERFACES WITH GENERICS (CONT'D)

```
import java.util.Iterator;
import java.lang.Iterable;

class Classroom implements Iterable<Student> {
    private Student[] students;

    Classroom(Student[] students) {
        this.students = students;
    }

    @Override
    public Iterator<Student> iterator() {
        return new CRIterator(students);
    }
    ...
}
```

```
...
class CRIterator implements Iterator<Student> {
    private int next = 0;

    @Override
    public boolean hasNext() {
        return next < arr.length;
    }

    @Override
    public Student next() {
        return new Student(arr[next++]);
    }
}
}
```

```
Student[] arrS = {new Student("Sam", 456), new Student("Paul", 123)};
ClassRoom classRoom = new ClassRoom(arrS);
for (Student student : classRoom) // now the code is indeed more elegant
    System.out.println(Student.toString());
```

DIAMOND OPERATOR

Reference type parameter must match the type parameter used at instantiation

```
Pair<Integer> intPair = new Pair<Integer>(2, 4);
```

Since Java 7, the compiler can infer the instantiation type parameter with <>

```
Pair<Integer> intPair = new Pair<>(2, 4);
```

By default `T` is a placeholder for an unbounded type

- ▶ There are no constraints on the type we can substitute to `T`
- ▶ The compiler safely assumes that `T` is replaced by `Object`

```
public class Pair<T> { // we want to use Person here
    private T first, second;

    public Pair(T first, T second) {
        this.first = first;
        this.second = second;
    }

    public T getFirst() { return first.getName(); } // compiler error
    public T getSecond() { return second; } // method not found for T
}
```

BOUNDED TYPE PARAMETER

We can define constraints on the types we can substitute to `T`

By specifying an **upper bound**

- We can instantiate `T` with `S1`, `S2`, ..., `Sk` or one of their subclasses

```
class Clazz<T extends S1 & S2 & ... & Sk>
```

By specifying a **lower bound**

- We can instantiate `T` with `S` or one of its superclasses

```
class Clazz<T super S>
```

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10
- 11

GENERICS AND INHERITANCE

*

GENERIC SUBTYPING

We should pay attention when we apply the substitution principle

- ▶ Since Student is a subtype of Person
- ▶ But Pair<Student> **is not** a subtype of Pair<Person>

```
Student s = new Student("Sam", 123);           1
Person p = s; // ok                             2
                                                3
Pair<Student> pairS = new Pair<>(sam, new Student("Paul", 456)); 4
Pair<Person> pairP = pairS; // compiler error    5
```

In general,Clazz<T> and Clazz<S> are not comparable in terms of subtyping

ARRAY SUBTYPING

We should pay attention when we apply the substitution principle

- ▶ Since Student is a subtype of Person
- ▶ And Student[] is a subtype of Person[]

```
Student s = new Student("Sam", 123);      1
Person p = s; // ok                        2
                                           3
Student[] arrS = {sam, new Student("Paul", 456)}; 4
Person[] arrP = arrS; // ok                5
```

WILDCARDS

With **wildcards** we can express (the lack of) constraints when using generic types

- ▶ `Clazz<?>` means `Clazz` of unknown (unbounded)
- ▶ The type of `?` is unknown, hence it is considered as `Object`

```
Student s = new Student("Sam", 123);           1
Person p = s; // ok                             2
                                                3
Pair<Student> pairS = new Pair<>(sam, new Student("Paul", 456)); 4
Pair<?> pairP = pairS; // ok                     5
```

... but we cannot call `Person` methods, like `pairP.getFirst().getName()`

WILDCARDS (CONT'D)

With **wildcards** we can express (the lack of) constraints when using generic types

- ▶ `Clazz<? extends T>` means Clazz of `T` or subclass
- ▶ `Clazz<? super T>` means Clazz of a `T` or superclass

```
<T extends Person> void printPair(Pair<T> p) {  
    System.out.println(p.getFirst().getName());  
    System.out.println(p.getSecond().getName());  
}
```

```
void printPair(Pair<? extends Person> p) {  
    System.out.println(p.getFirst().getName());  
    System.out.println(p.getSecond().getName());  
}
```

Analogous to type parameters, but less cumbersome

- ▶ We do not need class and method type parameter declarations

WILDCARDS (CONT'D)

With **wildcards** we can express (the lack of) constraints when using generic types

- ▶ `Clazz<? extends T>` means `Clazz` of `T` or subclass
- ▶ `Clazz<? super T>` means `Clazz` of a `T` or superclass

```
Student s = new Student("Sam", 123);           1
Person p = s; // ok                             2
                                                3
Pair<Student> pairS = new Pair<>(sam, new Student("Paul", 456)); 4
Pair<? extends Person> pairP = pairS; // ok      5
```

Now we can call `Person` methods, like `pairP.getFirst().getName()`

Q + A

